Mock 세로 슬라이스 개발 기록

프로젝트: 빙의했더니 영지 멸망 3년 전

기록 기준일: 2026년 10월 30일

개발 범위: Godot와 ASP.NET Core, SQLite를 연결한 Act 1~4 Mock 완주 흐름

1. 문서 목적

이 문서는 최초 개발 환경 구성부터 Mock 세로 슬라이스 완성까지의 기술적 판단, 구현 과정, 검증 근거와 문제 해결 경험을 기록한다. 단순 작업 일지가 아니라 향후 기능 확장과 포트폴리오 작성에 재사용할 수 있는 근거 문서를 목표로 한다.

2. 프로젝트 목표와 첫 번째 위험

최종 게임은 자유 대화, 고정 선택지, 영지 상태 계산, NPC 관계, 농사 미니게임, 제한시간 위기와 직업 엔딩을 포함한다. 그러나 이 요소를 한꺼번에 만들면 클라이언트와 서버의 계약이 맞지 않는 문제를 늦게 발견할 가능성이 높다.

따라서 첫 목표를 다음 한 줄로 제한했다.

> 타이틀 → NPC 대화 1회 → 선택지 1회 → 결과 계산 → 고정 직업 엔딩

이 범위는 화면, HTTP JSON 통신, 서버 상태 머신, 데이터 저장, 규칙 적용과 재시작을 한 번에 검증할 수 있는 최소 단위다. 실제 OpenAI API와 그래픽 작업은 이 흐름이 안정된 뒤로 미뤘다.

3. 아키텍처 원칙

3.1 서버 권위 구조

클라이언트는 플레이어 입력, 선택 ID와 농사 행동 결과만 전송한다. 능력치 변화량, 농사 점수, 진행 가능 여부와 엔딩은 서버가 결정한다. 이 구조는 클라이언트 조작을 줄이고 향후 콘텐츠가 늘어도 규칙을 한곳에서 관리하게 한다.

3.2 Mock과 Live의 동일 계약

Mock 모드도 실제 API와 같은 JSON 계약을 사용한다. 향후 OpenAI Provider를 추가할 때 Godot 화면과 진행 로직을 바꾸지 않고 서버 내부 구현만 교체할 수 있다.

3.3 데이터 기반 선택 규칙

각 Act의 장면, 대사, 선택지, 상태 변화와 직업 적성 변화는 content/acts/act*_mock.json에 정의했다. 새로운 선택지를 추가할 때 핵심 C# 코드를 반복 수정하지 않는 기반이다.

3.4 이벤트 기록과 중복 방지

각 세션은 진행 상태와 함께 append-only 게임 이벤트를 저장한다. 대화와 선택 요청은 UUID 형식의 idempotencyKey를 요구한다. 같은 키가 재전송되면 이미 저장된 응답을 돌려주므로 네트워크 재시도로 상태가 두 번 변하지 않는다.

4. 개발 과정

4.1 서버 도메인과 SQLite

다음 데이터를 SQLite에 저장하도록 구성했다.

- GameSession: Act, 턴, 진행 단계, 핵심 상태, 선택 결과와 엔딩
- GameEvent: 요청 순서, 이벤트 유형, 응답 JSON, 중복 요청 키
- NpcState: 신뢰, 두려움, 의심, 대화 요약
- TraitScore: 플레이 성향 점수
- CareerAffinity: 직업 적성과 마지막 변경 Act

서버 시작 시 개발용 데이터베이스를 생성하고, 세션 복구 API로 현재 상태와 완료된 엔딩을 다시 가져올 수 있게 했다.

4.2 Mock 게임 규칙

초기 상태는 군사 20, 재정 30, 정통성 25, 봉인 준비 10이다. 에드윈과 한 번 대화한 뒤 세 선택지를 제공한다.

선택	핵심 변화	직업 적성 변화
기사단 재건	군사 +20, 재정 -10	전투 +8, 지도력 +3
봉화 수리	봉인 +20, 재정 -10	마법 +8, 지도력 +3
주민 대피로 정비	정통성 +20, 재정 -5	지도력 +8, 외교 +3

Act 1~2 선택 결과는 다음 Act를 반환하며 상태와 적성을 누적한다. Act 3에서는 마르타와 농사 준비 방향을 정한 뒤 장애물 제거, 씨앗 배치, 물 배분과 작업 방식을 제출한다. 서버는 장애물 1개당 10점, 올바른 씨앗 1칸당 20점으로 계산하고 선택한 방식에 맞는 상태와 적성을 적용한다.

농사가 끝나면 Act 4 균열 발생 장면으로 이어진다. 최종 선택은 정면 방어, 봉인식, 외부 지원, 주민 대피 네 가지이며 20초 제한시간이 적용된다. 시간 초과는 주민 대피로 처리한다. 서버는 JSON에 정의된 선택별 필수 상태와 준비 기준으로 영지의 운명과 직업 엔딩을 결정한다.

4.3 Godot 화면 흐름

한 개의 메인 씬 안에서 타이틀, 대화, 선택지, 농사, 제한시간 최종 선택과 엔딩 화면을 전환한다. 선택 응답에 nextAct가 있으면 NPC와 장면을 갱신하고, nextActivity가 있으면 농사 화면을 표시한다. 최종 선택 버튼을 누르는 즉시 타이머를 멈춰 서버 응답 대기 시간이 제한시간에 포함되지 않게 했다. 요청 중에는 입력과 버튼을 잠가 중복 제출을 막고, 엔딩의 다시 시작 버튼은 새 세션을 생성한다.

헤드리스 실행 인자 --smoke-test는 네 Act와 성공 엔딩, 재시작을 자동으로 두 회차 실행한다. --smoke-timeout은 실제 Godot Timer 신호를 짧게 발생시켜 자동 대피 엔딩과 재시작을 확인한다.

이어하기는 Godot의 user:// 설정에 마지막 세션 ID만 보관하고, 실제 Act, 단계, 상태, 장면과 엔딩은 서버의 SQLite에서 다시 읽는다. 복구 단계에 따라 대화 화면, 선택지, 농사 진행 상태 또는 완료 엔딩을 표시한다. 농사 중 제거한 장애물과 심은 씨앗의 인덱스, 물 배분 대상과 작업 방식도 이벤트 체크포인트로 저장해 마지막 조작 상태를 복원한다.

5. API 흐름

```
POST /api/v1/sessions
  ↓ sessionId와 에드윈 장면
POST /api/v1/sessions/{id}/dialogues
  ↓ Mock 대사와 서사 피드백
GET /api/v1/sessions/{id}/choices
  ↓ 선택지 3개
POST /api/v1/sessions/{id}/choices
  ↓ 적용 변화, 다음 Act 또는 농사 활동
POST /api/v1/sessions/{id}/farming-progress
  ↓ 농사 조작 체크포인트 저장
POST /api/v1/sessions/{id}/farming
  ↓ 서버 점수·보상 계산, Act 4 장면
GET /api/v1/sessions/{id}/choices
  ↓ 서버가 저장한 Act 4 남은 제한시간
POST /api/v1/sessions/{id}/choices
  ↓ 최종 상태 판정과 직업 엔딩
GET /api/v1/sessions/{id}
  ↓ 현재 단계, 장면, 활동과 완료 엔딩 복구
```

오류 응답은 code, message, retryable, requestId를 가진 공통 JSON 구조를 사용한다.

6. 검증 결과

6.1 빌드와 실행

- .NET 서버 빌드: 경고 0, 오류 0
- Godot 4.5.1 프로젝트 파싱과 실행 성공
- /health: 서버와 SQLite 연결 정상

6.2 정상 경로

- Act 1~2 선택 후 엔딩 없이 다음 Act로 전환하고 누적 상태 유지
- Act 3 준비 선택 후 농사 활동으로 전환, 최대 작업 결과 90점 계산
- 물 배분과 작업 방식에 따른 상태·적성 보상 적용
- Godot 클라이언트에서 네 Act, 농사, 봉인 성공 엔딩과 재시작을 두 회차 연속 완주
- 4개 최종 선택지와 20초 제한시간 응답 확인
- Godot Timer 시간 초과 시 주민 대피 자동 선택과 재시작을 두 회차 연속 완주

- 봉인 준비 성공은 self_saved, 시간 초과는 people_evacuated로 판정
- 서로 다른 Godot 프로세스 사이에서 대화 전, 선택 대기, 농사 대기 상태 복구
- 완료 세션을 다시 열어 기존 직업 엔딩 복구
- 기사단 선택 후 군사 20에서 40으로 변경
- 봉화 선택 후 봉인 10에서 30으로 변경
- 대피로 선택 후 정통성 25에서 45로 변경

6.3 안정성 경로

- 같은 idempotencyKey 재전송 시 동일 응답 반환 및 중복 상태 변화 없음
- 허용 범위를 벗어난 농사 행동 횟수는 INVALID_FARMING_RESULT로 거부
- 빈 자유 대화는 HTTP 400과 INVALID_TEXT로 거부
- 서버 프로세스 재시작 후 SQLite에서 완료 세션과 엔딩 복구
- game.db, bin, obj, .godot은 Git 제외 확인
- 실제 API 키, GitHub 토큰과 사용자 대화 데이터가 소스에 없음

6.4 영지 운명 자동 테스트

실제 MockGameService, 인메모리 SQLite와 현재 JSON 콘텐츠를 사용하는 테스트 러너를 추가했다. 각 시나리오는 Act 1부터 농사와 Act 4 최종 선택까지 진행하며, 응답 엔딩과 저장된 엔딩이 일치하는지도 확인한다.

테스트	기대 결과	실제 결과	판정
자력 생존-기사단	self_saved	self_saved	PASS
자력 생존-봉인식	self_saved	self_saved	PASS
외부 지원	external_aid	external_aid	PASS
피해 생존	damaged_survival	damaged_survival	PASS
시간 초과 대피	people_evacuated	people_evacuated	PASS
영지 멸망	destroyed	destroyed	PASS
시간 초과 선택 검증	INVALID_TIMEOUT_CHOICE	INVALID_TIMEOUT_CHOICE	PASS
중복 요청 방지	상태 1회 적용	상태 1회 적용	PASS
기존 엔딩 복구 호환	survivor / orthodox 기본값	survivor / orthodox 기본값	PASS
일시 오류 재시도	2회째 정상 응답	2회째 정상 응답	PASS
잘못된 대화 JSON	고정 대사 폴백 후 진행	고정 대사 폴백 후 진행	PASS
대화 생성 타임아웃	고정 대사 폴백 후 진행	고정 대사 폴백 후 진행	PASS
농사 중간 복구	조작 상태와 선택 복원	장애물·씨앗·물·방식 복원	PASS
Act 4 타이머 복구	재접속해도 기존 마감 유지	만료 후 자동 주민 대피	PASS
전설 위상 해금	준비·적성 조건 충족 시 legendary	기사단장·마탑주 해금	PASS

테스트	기대 결과	실제 결과	판정
B급 톤 해금	봉인·마법·제작 업적 결합	균열 봉합 농기구 명장	PASS
Live Provider 성공	openai 응답으로 선택 진행	1회 응답 후 AwaitingChoice	PASS
Live 일시 오류	1회 재시도 후 openai 응답	2회째 정상 응답	PASS
Live 영구 오류	즉시 고정 대사 폴백	1회 뒤 fixed_fallback	PASS
OpenAI HTTP 계약	Responses API 구조화 출력	JSON Schema 요청·응답 추출	PASS
OpenAI 한도 오류	불필요한 재시도 없음	1회 뒤 fixed_fallback	PASS
Live 직업 후일담	확정 판정을 유지한 표현 생성	직업명·후일담만 교체	PASS
직업명 규칙 위반	기본 직업명과 고정 후일담 복구	fixed_fallback	PASS
Mock 직업 후보 선택	서로 다른 후보 3개를 서버가 점수화	근거 2개를 쓴 2번 후보 선택	PASS
OpenAI 후일담 HTTP 계약	엔딩 사실 기반 구조화 출력	JSON Schema 요청·응답 추출	PASS
엔딩 화면 시각 검증	제목부터 하단 버튼까지 접근	세로 스크롤·재시작 정상	PASS

6.5 직업 적성 기반 엔딩

2026-10-30에 누적 직업 적성과 최종 업적을 직업 엔딩에 연결했다. 서버가 영지 운명과 별도로 careerRank, endingTone, 상위 적성 2개와 근거를 확정하며, 외부 AI 없이 JSON의 고정 직업 템플릿을 사용한다. 성공한 최종 행동은 직업군의 핵심 근거로 우선하고, 실패 결말은 누적 1순위 적성을 반영한다.

초기 전설 조건인 준비 상태 3개와 적성 30은 현재 콘텐츠에서 도달할 수 없었다. 실제 선택 조합을 계산해 성공한 최종 업적, 준비 상태 2개 이상과 핵심 적성 24 이상으로 조정했다. 의외·B급 톤은 위상과 분리하고 toneAchievements에 최종 행동, 영지 운명과 최소 적성 조합을 선언한다. 조건과 직업명은 JSON에 있으므로 같은 형식의 항목을 추가해 엔딩을 확장할 수 있다.

경로	운명 / 위상 / 톤 / 직업	결과
기사단 방어 전설	self_saved / legendary / prestige / 구국의 기사단장	PASS
봉인식 전설	self_saved / legendary / prestige / 세 봉화의 마탑주	PASS
일반 봉인식	self_saved / accomplished / orthodox / 북부 봉화술사	PASS
마법·제작 복합 업적	self_saved / accomplished / b_grade / 균열 봉합 농기구 명장	PASS
외교·제작 복합 업적	external_aid / accomplished / unexpected / 왕국 공인 농기구 외교관	PASS
피해 생존	damaged_survival / survivor / orthodox / 피난민의 길잡이	PASS
시간 초과 대피	people_evacuated / survivor / orthodox / 피난민의 길잡이	PASS

경로	운명 / 위상 / 톤 / 직업	결과
영지 멸망	destroyed / fallen / orthodox / 유랑 봉화 연구자	PASS

6.6 대화 생성 실패 복구

IDialogueProvider 인터페이스 아래 Mock과 OpenAI Provider를 분리했다. live 세션은 OpenAI Responses API를 선택하고, NPC 대사·감정·피드백·허용 태그·직업 근거·요약을 Structured Outputs의 JSON Schema로 요청한다. 반환값은 기존 DTO의 길이와 태그 허용 목록을 다시 검증하며 능력치, 성공 여부, 다음 Act와 엔딩 판정은 프롬프트와 Provider 양쪽에서 다루지 않는다.

일시적인 HTTP 429·5xx·시간 초과 또는 잘못된 출력만 설정된 횟수만큼 재시도한다. 인증·설정 오류와 결제·사용량 한도는 반복 호출해도 회복되지 않으므로 즉시 Act별 고정 대사로 전환한다. API 키와 모델은 소스가 아닌 OpenAI__ApiKey, OpenAI__Model 환경 설정으로만 받는다. 키와 모델이 없는 실제 서버의 live 세션에서도 외부 호출 없이 fixed_fallback, 시도 1회로 선택지 단계까지 진행했다.

구현 기준은 OpenAI 공식 Structured Outputs와 오류 처리 문서를 사용했다.

- https://developers.openai.com/api/docs/guides/structured-outputs?api-mode=responses
- https://developers.openai.com/api/docs/guides/error-codes

6.7 생성형 직업 후일담

서버가 영지 운명, 직업 위상, 엔딩 톤, 상위 적성과 업적을 먼저 확정한 뒤 별도의 후일담 Provider가 표시용 직업명과 3~5문장 에필로그만 만든다. 생성 직업명에는 서버가 고른 기본 직업명이 반드시 포함되어야 하므로 AI가 기사단장을 떠돌이처럼 낮추거나 다른 직업군으로 바꿀 수 없다. 길이, 문장 수와 기본 직업명 보존 조건을 통과하지 못하면 기존 직업명과 고정 후일담으로 복구한다.

Mock과 Live는 같은 엔딩 응답을 사용하며 generationSource, generationFallbackUsed, generationAttemptCount로 생성 출처를 추적한다. Godot 엔딩 화면에는 새 후일담 영역을 추가했다. OpenAI 요청은 직업명과 후일담 두 필드만 가진 Structured Outputs를 사용하고, 영지 운명이나 위상을 출력 필드로 제공하지 않는다.

자동 테스트에서 Live 생성 성공, 직업명 규칙 위반 폴백과 Responses API HTTP 계약을 확인했다. 실제 API 키 없이도 Mock Provider가 동일한 후일담 계약을 채우므로 오프라인 완주와 세션 복구는 유지된다.

후일담 Provider는 서로 다른 후보를 정확히 3개 반환하도록 확장했다. 서버는 기본 직업명 보존, 길이, 문장 수와 중복을 먼저 검증하고 영지 결과, 플레이 근거와 성향이 문장에 직접 반영된 정도를 점수화한다. 동점이면 먼저 생성된 후보를 선택하며, 후보 계약이 하나라도 깨지면 생성 시도를 무효 처리한다. 응답에는 후보 수, 선택 인덱스와 점수를 남겨 선택 과정을 추적할 수 있다. Mock 검증에서는 플레이 근거 2개를 사용한 2번 후보가 16점으로 선택됐고, 규칙 테스트 25개가 모두 통과했다.

실제 Godot 기본 창에서 처음 확인했을 때 후일담이 추가된 VBoxContainer의 전체 높이가 화면을 넘어 상단 제목이 잘렸다. 엔딩 루트를 ScrollContainer로 바꾸고 내부 콘텐츠를 상단 정렬했으며, 엔딩을 열 때마다 스크롤 위치를 0으로 초기화했다. 재검증에서는 제목이 처음부터 보였고 세로 스크롤로 결정적 기록과 다시 시작 버튼까지 접근해 새 세션을 정상 시작했다.

6.8 진행 중 체크포인트 복구

농사 조작은 제출 전에도 전용 API로 직렬 저장한다. 별도 테이블을 추가하는 대신 기존 append-only GameEvent에 체크포인트를 기록해 이전 SQLite 파일과의 호환성을 유지했다. Act 4는 최초 선택지 조회 시 서버가 절대 마감 시각을 한 번만 저장하고 이후 남은 시간을 계산한다. 따라서 앱을 다시 열어도 20초가 초기화되지 않으며, 마감 뒤 도착한 선택도 서버가 주민 대피로 교정한다.

자동 테스트 25개와 실제 Godot 재실행 스모크 테스트를 통과했다. 농사 재실행에서는 장애물 〔0, 2〕, 씨앗 〔0, 3〕, 물 market, 방식 tools가 복원됐고, 제한시간 종료 뒤 재실행에서는 남은 시간 0.00과 people_evacuated 엔딩을 확인했다.

7. 문제와 해결

7.1 실행 환경의 로컬 포트 제한

자동화 환경 안에서는 로컬 포트 접근이 분리돼 서버와 클라이언트가 서로 보이지 않았다. 실제 로컬 실행 권한이 있는 환경에서 서버와 Godot을 실행해 종단간 통신을 검증했다. 코드 문제와 실행 격리 문제를 분리해 진단한 경험이다.

7.2 SQLite 네이티브 의존성 보안 경고

최초 복원에서 SQLitePCLRaw.lib.e_sqlite3 2.1.11에 대한 고위험 취약점 경고가 발생했다. 공식 보안 권고를 확인하고 SQLite 3.53.4 이상을 포함하는 SQLitePCLRaw.bundle_e_sqlite3 3.0.5를 직접 참조했다. 재빌드 결과 경고 0, 오류 0을 확인했다.

참고 자료:

- https://github.com/advisories/GHSA-2m69-gcr7-jv3q
- https://www.nuget.org/packages/SQLitePCLRaw.bundle_e_sqlite3

7.3 네트워크 재시도의 이중 적용 위험

진행 요청이 성공했지만 응답이 끊기면 클라이언트가 같은 요청을 재전송할 수 있다. 이벤트에 중복 요청 키와 전체 응답을 저장하고 같은 키에는 저장된 응답을 반환하도록 해 상태 변화가 한 번만 적용되게 했다.

8. 결과와 의미

이 작업으로 Godot UI, ASP.NET Core API, JSON 콘텐츠, SQLite 저장과 서버 규칙이 네 Act, 농사, 제한시간 최종 선택과 적성 기반 직업 엔딩까지 연결되는 것을 확인했다. 실제 AI와 그래픽이 없어도 한 판을 처음부터 끝까지 완주하고 다시 시작할 수 있다.

포트폴리오 관점에서 다음 역량을 증명하는 작업이다.

- 큰 기획을 가장 위험한 종단간 흐름으로 축소하는 범위 설계
- 클라이언트와 서버 책임 분리
- 데이터 기반 규칙 설계
- SQLite 기반 상태 및 이벤트 저장
- 중복 요청과 오류 계약을 고려한 API 안정성
- 실제 Godot 클라이언트를 포함한 종단간 자동 검증

- 진행 중 체크포인트와 서버 권위 제한시간 복구
- 플레이 근거를 요구하는 데이터 기반 위상·톤 해금 규칙
- Mock·OpenAI Provider 교체, 구조화 출력 검증과 오류별 폴백
- 서버 판정을 보존하는 생성형 직업명·후일담 표현 계층
- 후일담 후보 3개 검증·점수화와 선택 근거 추적
- 의존성 취약점 탐지와 해결

9. 한계와 다음 작업

현재 고정 콘텐츠 기반 Mock 게임은 Act 1부터 최종 엔딩까지 완주되며 OpenAI 대화와 직업 후일담 Provider 경계도 연결됐다. 다음 작업은 아래와 같다.

1. 로컬 환경 변수에만 API 키와 모델을 넣어 실제 Live 대화·후일담 품질과 지연 검증

Live 호출이 실패해도 현재 고정 대사와 고정 직업 엔딩으로 완주할 수 있어야 한다.

10. 재현 방법

서버 실행:

```
dotnet run --project server/Game.Api/Game.Api.csproj
```

Godot 실행:

```
/Applications/Godot.app/Contents/MacOS/Godot --path client
```

자동 스모크 테스트:

```
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-test
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-timeout
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-save
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-resume
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-save-farming-progress
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-save-final-timer
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-resume-timer
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-dialogue-transient
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-dialogue-invalid
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-dialogue-timeout
```

영지 운명 규칙 테스트:

```
dotnet run --project tests/Game.Rules.Tests/Game.Rules.Tests.csproj
```

PDF 재생성:

```
python3 tools/generate_development_report.py
```